كلية العلوم التطبيقية - ورزازات

ⵜⴰⵖⵍⵓⵜ ⵏ ⵜⵙⴰⵏⵙⵉⵡⵉⵏ - ⵡⵓⵔⵣⴰⵣⴰⵜ

FACULTE DES SCIENCES APPLIQUEES - OUARZAZATE

TRAVAUX PRATIQUES – 2

GESTION DES NOTES DES ÉTUDIANTS

JavaScript en pratique

Tableaux et objets • Validation des notes • Calcul de moyenne • Préparation de l'affichage mobile

Module	Développement mobile avec React Native
Filière	IAIL S5
Environnement	Node.js, VS Code ; Expo Go en prolongement facultatif
Professeur	Pr. OMARI Kamal
Année universitaire	2026–2027

Réalisé par : Abdelhafid Ait Lafki

Faculté des Sciences Appliquées – Ouarzazate

SOMMAIRE

01 Préparer son espace de travail

Étape 1 Vérifier Node.js et créer un dossier

Étape 2 Créer le fichier de travail

02 Créer le tableau des étudiants

Étape 3 Déclarer les données

Étape 4 Consulter le premier étudiant

03 Construire des fonctions

Étape 5 Vérifier les notes

Étape 6 Calculer la moyenne

04 Filtrer puis préparer l·affichage

Étape 7 Sélectionner les admis

Étape 8 Construire des lignes d·affichage

05 Comprendre le parcours des données

06 Correction intégrale : fichier notes.js

07 Exercices de compréhension corrigés

Étape 9 Comparer sans conversion automatique

Étape 10 Extraire des propriétés d·un objet

08 Vérifications finales et prolongement

Étape 11 Afficher les données sur mobile avec React Native

Étape 1 — Vérifier Node.js et créer un dossier

Ouvrez un terminal et saisissez les commandes suivantes :

```
node --version
mkdir tp2-notes
cd tp2-notes
```

```
Exemple de sortie pour node --version :
vXX.XX.X

Les commandes mkdir et cd ne produisent généralement pas de texte si elles réussissent.
```

Explication : Node.js est un environnement capable d'exécuter JavaScript en dehors du navigateur. La commande node --version (deux tirets avant « version ») permet de vérifier son installation. La version exacte dépend de l'installation.

Point de contrôle A

Le terminal affiche une version de Node.js. Si la commande n'est pas reconnue, installez une version LTS de Node.js et ouvrez un nouveau terminal.

Étape 2 — Créer le fichier de travail

Dans Visual Studio Code, créez le fichier notes.js à l'intérieur du dossier tp2-notes. Placez-y :

```
console.log("TP2 : gestion des notes");
```

Puis lancez le fichier dans le terminal :

```
node notes.js
```

```
TP2 : gestion des notes
```

Explication : console.log() affiche le texte dans la console. La commande node notes.js demande à Node.js d'exécuter le fichier notes.js.

03 — Créer le tableau des étudiants

Étape 3 — Déclarer les données

Remplacez le contenu de notes.js par :

```
const etudiants = [
  { id: 1, nom: "Amine", note: 16 },
  { id: 2, nom: "Youssef", note: 8 },
  { id: 3, nom: "Salma", note: 14 },
  { id: 4, nom: "Omar", note: 12 }
```

02 · Créer le tableau des étudiants

```
node notes.js
```

```
[
  { id: 1, nom: 'Amine', note: 16 },
  { id: 2, nom: 'Youssef', note: 8 },
  { id: 3, nom: 'Salma', note: 14 },
  { id: 4, nom: 'Omar', note: 12 }
]
```

Explication : etudiants est un tableau. Chaque élément est un objet ayant trois propriétés : id (identifiant), nom et note. const empêche de réaffecter la variable etudiants, sans rendre les objets immuables. La présentation exacte des objets peut varier légèrement selon la version de Node.js.

Étape 4 — Consulter et compter

Ajoutez ces instructions après la déclaration du tableau :

```
console.log("Premier :", etudiants[0].nom);
console.log("Effectif :", etudiants.length);
```

Exécution :

```
node notes.js
```

03 · Construire des fonctions

length donne le nombre d'éléments du tableau.

04 — Construire des fonctions

Étape 5 — Vérifier les notes

Ajoutez la fonction suivante :

```
function noteValide(note) {
  return Number.isFinite(note) &&
         note >= 0 && note <= 20;
}
console.log(noteValide(16));
console.log(noteValide(25));
```

Exécution :

```
node notes.js
```

```
true
false
```

Explication : La fonction accepte une note uniquement si elle est un nombre fini compris entre 0 et 20 inclus. 16 est valide ; 25 dépasse la note maximale autorisée.

Étape 6 — Calculer la moyenne

Ajoutez la fonction de calcul de moyenne et extrayez les notes avec map() :

```
function moyenne(notes) {
  if (notes.length === 0) return null;
  const total = notes.reduce(
    (somme, note) => somme + note, 0
  );
```

04 · Filtrer puis préparer l·affichage

Exécution :

```
node notes.js
```

```
Moyenne : 12.5
```

Explication : map() récupère les notes de chaque étudiant. reduce() les additionne en commençant à 0. La moyenne est la somme divisée par le nombre de notes. Si le tableau est vide, la fonction renvoie null pour signaler qu'il n'y a pas de moyenne. Ici : (16 + 8 + 14 + 12) / 4 = 12.5.

05 — Filtrer puis préparer l'affichage

Étape 7 — Sélectionner les admis

```
const admis = etudiants.filter(
  (e) => e.note >= 10
);
console.log("Admis :", admis.length);
```

Exécution :

```
node notes.js
```

```
Admis : 3
```

Explication : filter() conserve les étudiants dont la note est supérieure ou égale à 10. Trois étudiants sont admis ; Youssef, qui a 8/20, n'atteint pas le seuil de 10/20.

Étape 8 — Construire des lignes d'affichage

```
const lignes = etudiants.map((e) => {
  const decision = e.note >= 10 ? "Admis" : "Ajourné";
  return `${e.nom} - ${e.note}/20 - ${decision}`;
});
for (const ligne of lignes) {
  console.log(ligne);
}
```

Exécution :

```
node notes.js
```

```
Amine - 16/20 - Admis
Youssef - 8/20 - Ajourné
Salma - 14/20 - Admis
```

» si la note est au moins 10, sinon « Ajourné ». La boucle for affiche chaque ligne.

06 — Comprendre le parcours des données

etudiants	map()	reduce()
4 étudiants avec id, nom et note	Une liste de nombres	Somme des notes puis moyenne

Moyenne = somme des notes / effectif

Question accompagnée

Pourquoi ne pas calculer reduce() directement sur le tableau etudiants sans adapter sa fonction ?

Explication : Chaque élément de etudiants est un objet et non une note numérique. Il faut accéder à sa propriété note, par exemple avec e.note, dans la fonction d'accumulation.

Lien avec React Native

Explication : Dans un prochain chapitre, les chaînes du tableau lignes pourront être placées dans des composants Text. Dans ce TP, on valide d'abord les calculs dans la console.

07 — Correction intégrale : fichier notes.js

Le fichier final à conserver dans le dossier tp2-notes est le suivant :

```
const etudiants = [
  { id: 1, nom: "Amine", note: 16 },
  { id: 2, nom: "Youssef", note: 8 },
  { id: 3, nom: "Salma", note: 14 },
  { id: 4, nom: "Omar", note: 12 }
];

function noteValide(note) {
  return Number.isFinite(note) &&
    note >= 0 && note <= 20;
}

function moyenne(notes) {
  if (notes.length === 0) return null;
  const total = notes.reduce(
    (somme, note) => somme + note, 0
  );
  return total / notes.length;
}

const notes = etudiants.map((e) => e.note);
const admis = etudiants.filter((e) => e.note >= 10);
const lignes = etudiants.map((e) => {
  const decision = e.note >= 10 ? "Admis" : "Ajourné";
  return `${e.nom} - ${e.note}/20 - ${decision}`;
});

console.log("Moyenne :", moyenne(notes));
console.log("Admis :", admis.length);
for (const ligne of lignes) console.log(ligne);
```

Exécution dans le terminal :

```
node notes.js
```

07 · Exercices de compréhension corrigés

```
Youssef - 8/20 - Ajourné
Salma - 14/20 - Admis
Omar - 12/20 - Admis
```

Explication : Ce fichier rassemble les données, les fonctions de validation et de calcul, le filtrage des admis et la préparation des lignes destinées à l'affichage.

08 — Exercices de compréhension corrigés

1. Pourquoi utilise-t-on const pour le tableau ?

08 · Comparer sans conversion automatique

2. Quelle différence existe entre map() et filter() ?

Explication : map() transforme chaque élément ; filter() conserve ceux qui vérifient une condition.

3. Pourquoi tester un tableau vide ?

Explication : Pour ne pas calculer une moyenne en divisant par zéro.

09 · Extraire des propriétés d·un objet

5. Si Salma passe de 14 à 18, combien d'admis obtient-on ?

Explication : Toujours 3, car Salma était déjà admise. La nouvelle moyenne devient 13.5.

Étape 9 — Comparer sans conversion automatique

Observez les deux expressions :

```
console.log(10 == "10");   // true
console.log(10 === "10");  // false
```

Exécution :

```
node notes.js
```

```
true
false
```

Explication : L'opérateur == peut convertir les types avant de comparer. L'opérateur === vérifie à la fois le type et la valeur. Pour tester une note numérique, privilégiez une comparaison stricte et vérifiez que la donnée est bien un nombre.

Étape 10 — Extraire des propriétés d'un objet

La déstructuration permet de récupérer plusieurs propriétés en une instruction :

```
const { nom, note } = etudiants[0];
console.log(`${nom} : ${note}/20`);
```

Résultat :

```
Amine : 16/20
```

Explication : nom et note sont extraits du premier objet du tableau, etudiants[0], puis affichés dans une chaîne de texte.

✓ Le programme s'exécute avec node notes.js sans message d'erreur.

✓ Les quatre étudiants sont représentés sous forme d'objets.

✓ La moyenne est 12.5 et le nombre d'admis est 3.

✓ Le programme affiche quatre lignes lisibles et traite le cas d'un tableau vide.

À rendre

Le fichier notes.js, une capture des résultats du terminal et les réponses aux questions.

Étape 11 — Prolongement : afficher les données sur mobile

Cette démonstration facultative réutilise View et Text du TP1 et introduit l'idée d'afficher une liste. Dans le projet Expo créé au TP1 (modèle avec App.tsx), remplacez temporairement le contenu de App.tsx par :

```
import { View, Text, StyleSheet } from "react-native";

const etudiants = [
  { id: 1, nom: "Amine", note: 16 },
  { id: 2, nom: "Youssef", note: 8 },
  { id: 3, nom: "Salma", note: 14 },
  { id: 4, nom: "Omar", note: 12 }
];

const total = etudiants.reduce((s, e) => s + e.note, 0);
const moyenne = total / etudiants.length;

export default function App() {
  return (
    <View style={styles.ecran}>
      <Text style={styles.titre}>Notes des étudiants</Text>
      <Text style={styles.moyenne}>Moyenne : {moyenne}/20</Text>
      {etudiants.map((e) => (
        <Text key={e.id} style={styles.ligne}>
          {e.nom} : {e.note}/20
        </Text>
      ))}
    </View>
  );
}

const styles = StyleSheet.create({
  ecran: {
    flex: 1, justifyContent: "center", padding: 26,
    backgroundColor: "#F2F7FD"
  },
  titre: {
    fontSize: 24, fontWeight: "bold", color: "#00235C",
    marginBottom: 16
  },
  moyenne: { fontSize: 19, marginBottom: 20, color: "#003796" },
  ligne: { fontSize: 17, paddingVertical: 8, color: "#00235C" }
});
```

Lancement de l'application Expo

Dans le dossier du projet Expo existant, lancez :

```
npx expo start
```

Puis ouvrez le projet avec Expo Go, ou utilisez un émulateur déjà configuré.

Résultat visuel attendu

```
Notes des étudiants
Moyenne : 12.5/20
Amine : 16/20
Youssef : 8/20
Salma : 14/20
Omar : 12/20
```

Explication : L'écran présente le titre « Notes des étudiants », la moyenne 12.5/20 puis les quatre lignes de notes. La propriété key aide React à distinguer les éléments répétés.